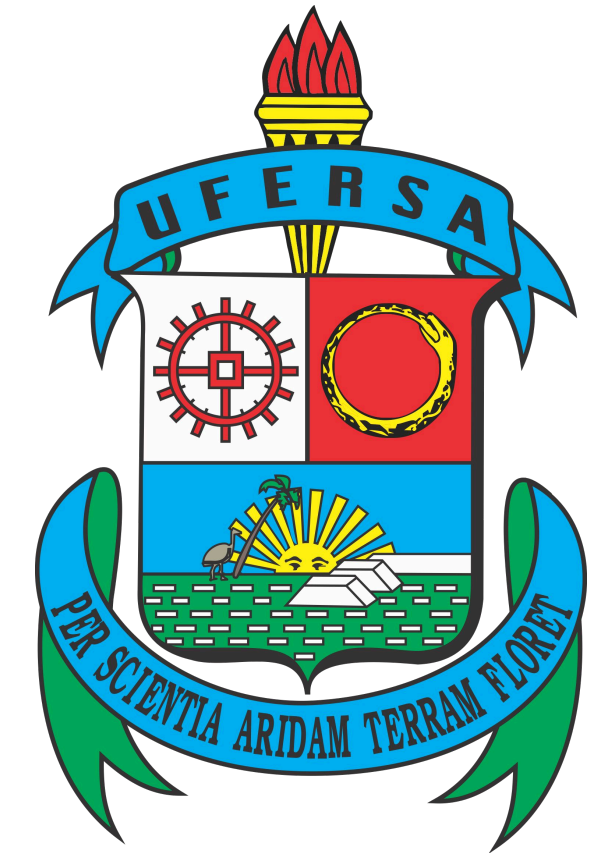


Programação Orientada a Objetos

Capítulo 5: Instruções de Controle: Parte 2

Ruan Pablio Barbosa Claudino



Objetivos da aula

Compreender a repetição por contador

Os quatro elementos necessários: variável de controle, valor inicial, incremento e condição de continuação do loop.

Aplicar a instrução for

Expressar a repetição controlada por contador em uma única linha de código, no cabeçalho do for.

Usar a instrução do...while

Repetir um bloco de instruções garantindo que o corpo seja executado pelo menos uma vez.

Reconhecer a precedência de operadores

Aplicar corretamente a ordem de avaliação e associatividade dos operadores Java vistos até agora.

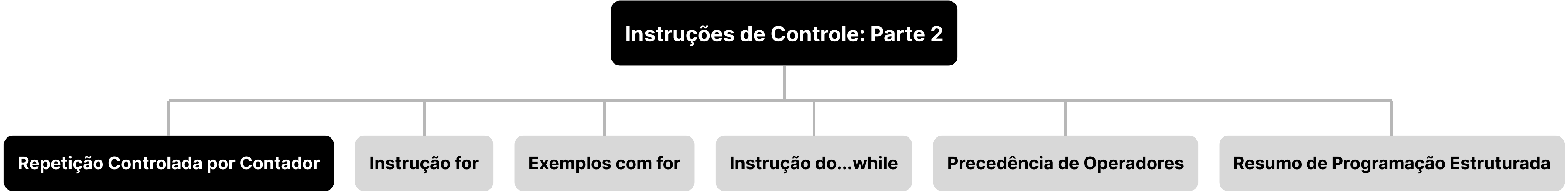
Resumir a programação estruturada

Revisar como sequência, seleção e repetição se combinam para formar qualquer programa estruturado.

Introdução

"Manter a coisa simples" é um bom conselho para a maior parte do código que você escreverá.

Mapa da Aula



Repetição Controlada por Contador

Realizar a repetição controlada por contador requer uma variável de controle, seu valor inicial, um incremento por iteração e uma condição de continuação do loop.

Variável de controle

Também chamada contador do loop, reserva espaço na memória para controlar a repetição.

Valor inicial

O valor com que a variável de controle começa antes da primeira iteração do loop.

Incremento

A quantidade pela qual a variável de controle é modificada a cada passagem pelo loop.

Condição de continuação

Testada a cada iteração, determina se o loop deve continuar executando.

Repetição por Contador: while

```
// Figura 5.1: WhileCounter.java
// Repetição controlada por contador com a instrução de repetição while.

public class WhileCounter
{
    public static void main(String[] args)
    {
        int counter = 1; // declara e inicializa a variável de controle

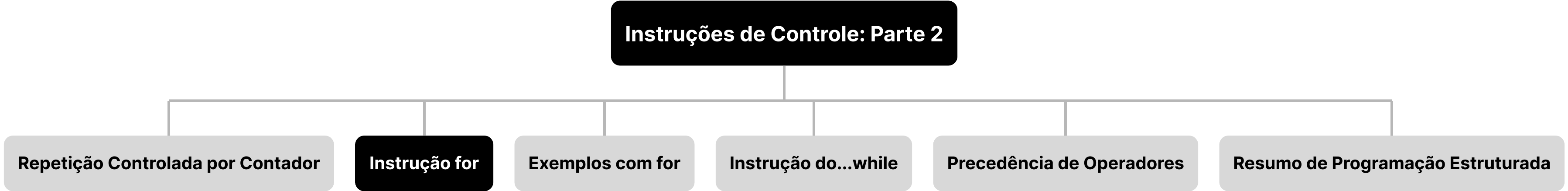
        while (counter ≤ 10) // condição de continuação do loop
        {
            System.out.printf("%d ", counter);
            ++counter; // variável de controle de incremento
        }

        System.out.println();
    }
} // fim da classe WhileCounter
```

SAÍDA

1 2 3 4 5 6 7 8 9 10

Mapa da Aula



Repetição por Contador: for

```
// Figura 5.2: ForCounter.java
// Repetição controlada por contador com a instrução for.

public class ForCounter
{
    public static void main(String[] args)
    {
        // o cabeçalho da instrução for inclui inicialização,
        // condição de continuação do loop e incremento
        for (int counter = 1; counter ≤ 10; counter++)
            System.out.printf("%d ", counter);

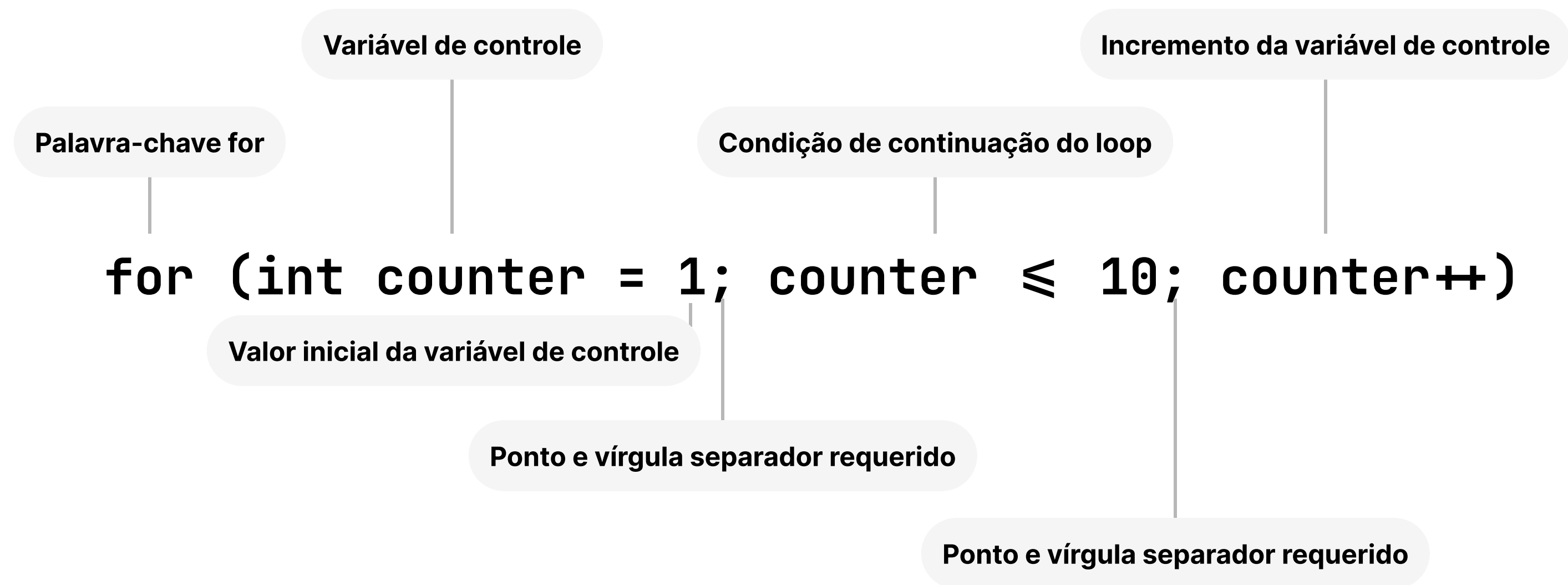
        System.out.println();
    }
} // fim da classe ForCounter
```

SAÍDA

1 2 3 4 5 6 7 8 9 10

Componentes do Cabeçalho da Instrução for

O cabeçalho de for "faz tudo": **especifica cada item necessário para a repetição** controlada por contador com uma variável de controle.



Formato Geral da Instrução for

O cabeçalho for nomeia a variável de controle e especifica, em uma única linha, seu valor inicial, a condição de continuação do loop e o incremento.

FORMATO GERAL

```
for (inicialização; condiçãoDeContinuaçãoDoLoop;  
    incremento)  
    instrução
```

EQUIVALENTE COM WHILE

```
inicialização;  
while (condiçãoDeContinuaçãoDoLoop)  
{  
    instrução  
    incremento;  
}
```

Escopo da Variável de Controle

Se a expressão inicialização no cabeçalho for declara a variável de controle, ela pode ser utilizada somente nessa instrução for — não existirá fora dela. Esse uso restrito é conhecido como **escopo da variável**.

Expressões Opcionais no Cabeçalho

Todas as três expressões em um cabeçalho for são opcionais. Se a condiçãoDeContinuaçãoDoLoop for omitida, o Java assume que ela é sempre verdadeira, **criando um loop infinito**.

```
counter = counter + 1  
counter += 1  
++counter  
counter++
```

São expressões de incremento equivalentes em uma instrução for — muitos programadores preferem counter++ por ser mais conciso.

Diagrama de Atividades UML para for

O diagrama deixa claro que a inicialização ocorre uma vez antes do teste, e que o incremento ocorre a cada passagem pelo loop, **depois que a instrução do corpo executa**.

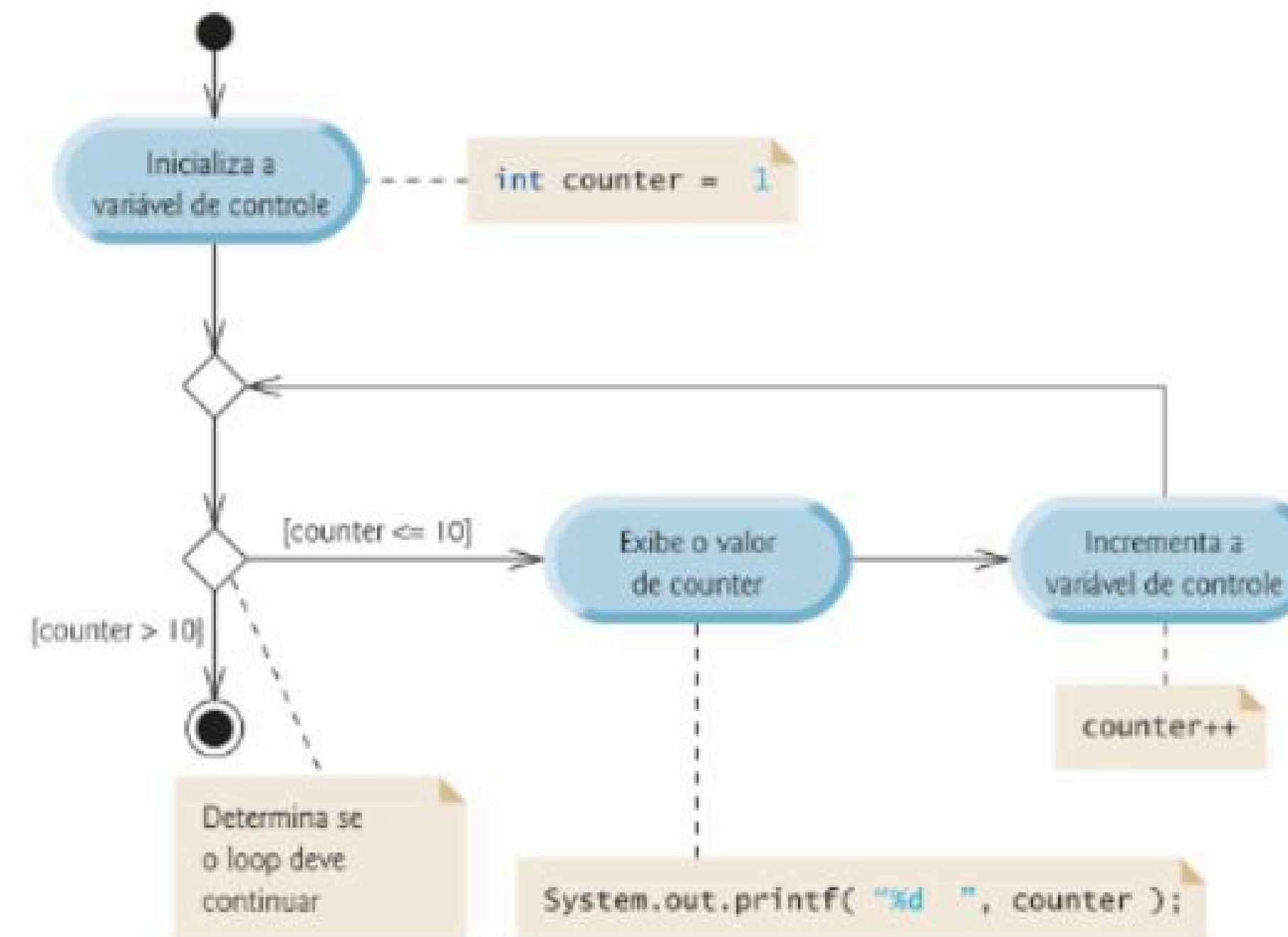
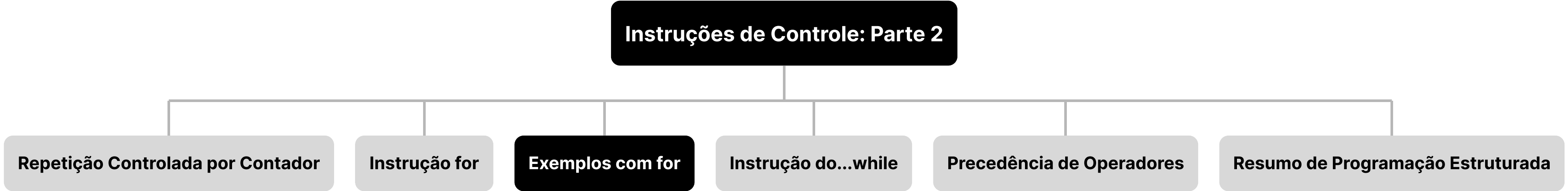


Figura 5.4 | Diagrama de atividades UML para a instrução for na Figura 5.2.

Mapa da Aula



Exemplos com a Estrutura for

Em cada caso, apenas o cabeçalho for muda — observe a alteração no operador relacional para os loops que decrementam.

a) De 1 a 100, incrementos de 1

```
for (int i = 1; i ≤ 100; i++)
```

b) De 100 a 1, decrementos de 1

```
for (int i = 100; i ≥ 1; i--)
```

c) De 7 a 77, incrementos de 7

```
for (int i = 7; i ≤ 77; i += 7)
```

d) De 20 a 2, decrementos de 2

```
for (int i = 20; i ≥ 2; i -= 2)
```

e) Valores 2, 5, 8, 11, 14, 17, 20

```
for (int i = 2; i ≤ 20; i += 3)
```

f) Valores 99, 88, ..., 11, 0

```
for (int i = 99; i ≥ 0; i -= 11)
```

Somando os Inteiros Pares de 2 a 20

```
// Figura 5.5: Sum.java
// Somando inteiros com a instrução for.

public class Sum
{
    public static void main(String[] args)
    {
        int total = 0;

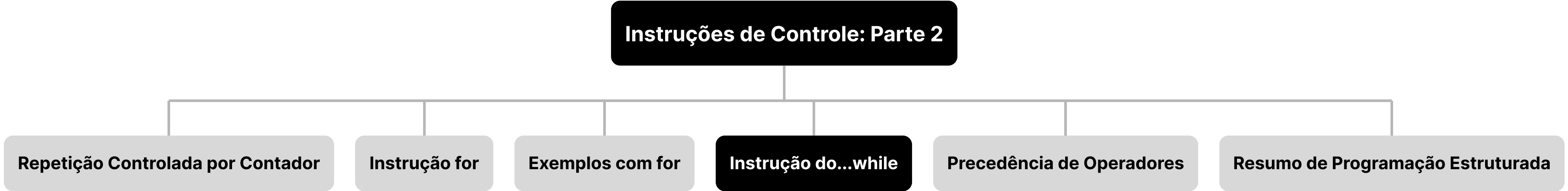
        // total de inteiros pares de 2 a 20
        for (int number = 2; number ≤ 20; number += 2)
            total += number;

        System.out.printf("Sum is %d\n", total);
    }
} // fim da classe Sum
```

SAÍDA

Sum is 110

Mapa da Aula



Instrução de Repetição do...while

É semelhante à instrução while, mas **testa a condição de continuação do loop depois de executar o corpo** — portanto, o corpo sempre executa pelo menos uma vez.

Repetição com a Instrução do...while

```
// Figura 5.7: DoWhileTest.java
// instrução de repetição do...while.

public class DoWhileTest
{
    public static void main(String[] args)
    {
        int counter = 1;

        do
        {
            System.out.printf("%d ", counter);
            ++counter;
        } while (counter ≤ 10); // fim da instrução do...while

        System.out.println();
    }
} // fim da classe DoWhileTest
```

SAÍDA

1 2 3 4 5 6 7 8 9 10

Chaves na Instrução do...while

Sem chaves, um leitor pode interpretar erroneamente a última linha como uma instrução while separada — **por isso o corpo é normalmente escrito com chaves.**

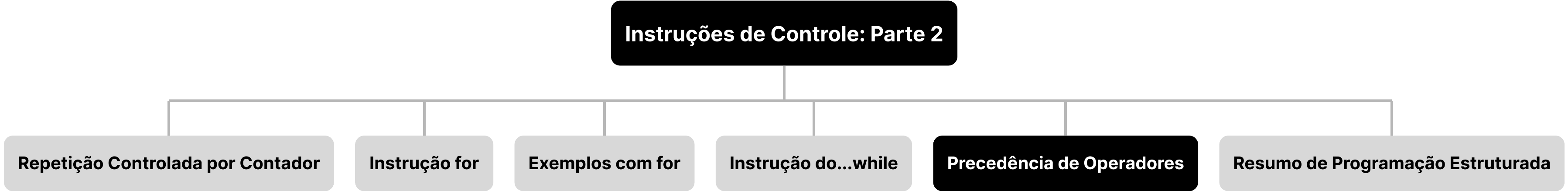
SEM CHAVES (CONFUSO)

```
do
    instrução
while (condição);
```

COM CHAVES (RECOMENDADO)

```
do
{
    instrução
} while (condição);
```

Mapa da Aula

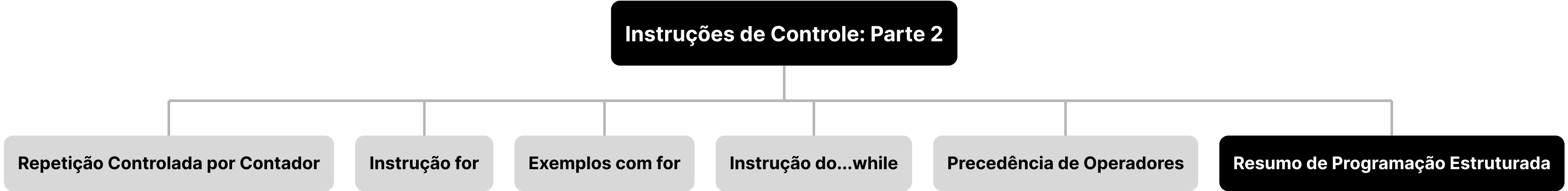


Precedência e Associatividade de Operadores

Operadores em ordem decrescente de precedência.

Operadores	Associatividade	Tipo
<code>++</code> <code>--</code>	da direita para a esquerda	unário pós-fixo
<code>++</code> <code>--</code> <code>+</code> <code>-</code> <code>!</code> <code>(tipo)</code>	da direita para a esquerda	unário pré-fixo
<code>*</code> <code>/</code> <code>%</code>	da esquerda para a direita	multiplicativo
<code>+</code> <code>-</code>	da esquerda para a direita	aditivo
<code><</code> <code>≤</code> <code>></code> <code>≥</code>	da esquerda para a direita	relacional
<code>=</code> <code>≠</code>	da esquerda para a direita	igualdade
<code>&</code>	da esquerda para a direita	E lógico booleano
<code>^</code>	da esquerda para a direita	OU exclusivo lógico booleano
<code> </code>	da esquerda para a direita	OU inclusivo lógico booleano
<code>&&</code>	da esquerda para a direita	E condicional
<code> </code>	da esquerda para a direita	OU condicional
<code>?:</code>	da direita para a esquerda	ternário condicional
<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code>	da direita para a esquerda	atribuição

Mapa da Aula



Três Formas de Controle

São necessárias apenas **três formas de controle para implementar qualquer algoritmo**, combinadas por empilhamento e aninhamento.

Sequência

É trivial: liste as instruções na ordem em que devem executar.

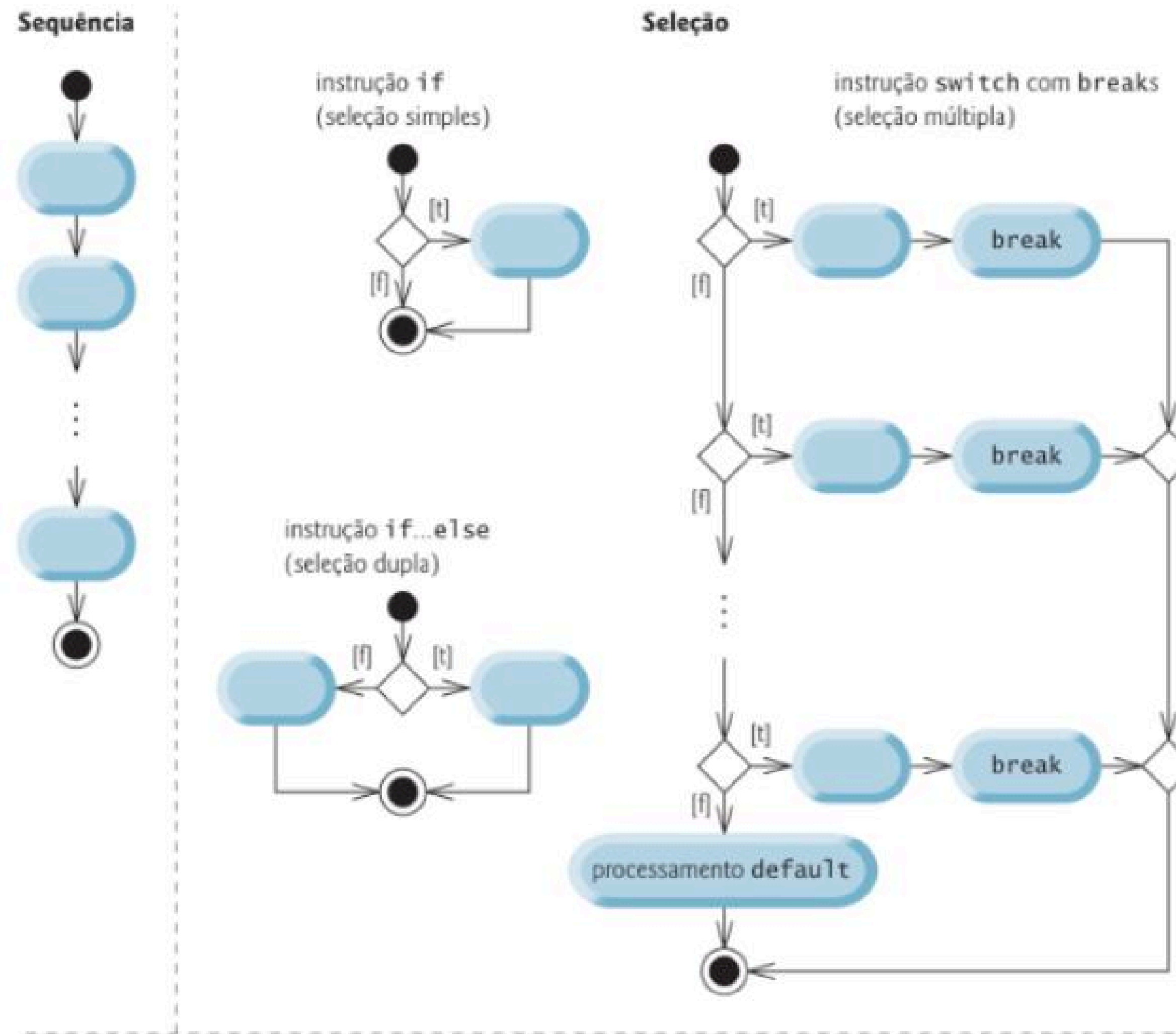
Seleção

Implementada por if (única), if...else (dupla) ou switch (múltipla).

Repetição

Implementada por while, do...while ou for.

Instruções de Controle



Instruções de Controle

As instruções while, do...while e for compartilham a mesma estrutura — muda apenas a posição do teste e do incremento.

